

Agentes de inteligencia artificial y MCP

Conceptos, integración y aplicaciones en la industria

Joaquín F. Sánchez

Octubre de 2026

Al finalizar, podremos explicar cómo un agente utiliza herramientas y cómo MCP conecta esas capacidades con sistemas empresariales.

- ➊ Concepto y ciclo de un agente.
- ➋ Arquitectura y capacidades de MCP.
- ➌ Aplicaciones industriales y criterios de implementación.
- ➍ Demostración con el agente de inventario.

Definición operativa

Un agente percibe información del entorno y selecciona acciones para alcanzar un objetivo dentro de unas restricciones.

- **Entorno:** usuarios, documentos, dispositivos o aplicaciones.
- **Percepción:** mensajes, mediciones y resultados de consultas.
- **Acción:** responder, solicitar datos o ejecutar una operación.
- **Objetivo:** criterio que permite evaluar el resultado.

Los agentes pueden usar reglas, búsqueda, aprendizaje o modelos de lenguaje. Esta presentación se concentra en agentes con modelos de lenguaje.

Agente basado en un modelo de lenguaje

- El **modelo** interpreta la solicitud y propone el siguiente paso.
- Las **instrucciones** definen la tarea y sus límites.
- Las **herramientas** consultan datos o ejecutan operaciones.
- El **ejecutor** administra las llamadas y sus resultados.
- El **estado** conserva el contexto necesario entre pasos.

La capacidad de actuar depende de las herramientas y permisos que la aplicación ofrece.

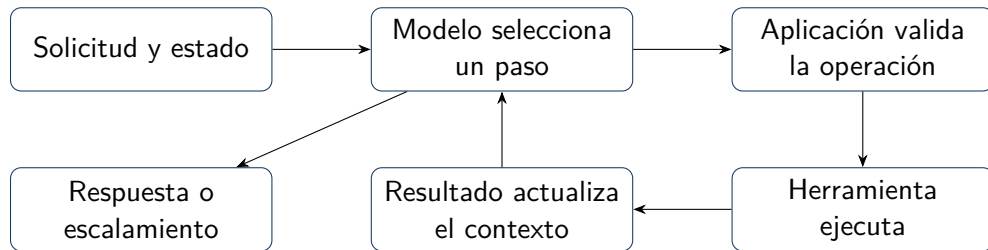
Fuente: OpenAI, Agent definitions. Consulta: 7 de octubre de 2026.

Chat, flujo de trabajo y agente

Enfoque	Selección de pasos	Ejemplo
Chat	Genera una respuesta a la entrada	Explica qué es un inventario
Flujo definido	El programa establece la secuencia	Cada noche calcula faltantes
Agente	El modelo selecciona pasos dentro de un marco	Consulta productos y propone reposición

Una interfaz de chat puede contener un agente. Un sistema también puede combinar pasos definidos con decisiones del modelo.

Ciclo de ejecución



- Un límite de pasos evita ciclos indefinidos.
- Un error puede exigir aclaración, reintento o intervención humana.

Fuente: OpenAI, Running agents. Consulta: 7 de octubre de 2026.

Herramientas y contratos

Una herramienta necesita nombre, descripción, argumentos y un resultado interpretable.

Ejemplo de contrato

```
consultar_stock(producto: str)
```

Entrada: identificador P001.

Salida: existencia, mínimo, objetivo y precio, o un error explícito.

- La función valida los datos antes de operar.
- El servicio calcula cantidades y costos con reglas deterministas.
- El agente interpreta el resultado para responder al usuario.

Fuente: OpenAI, Agents SDK quickstart. Consulta: 7 de octubre de 2026.

Contexto y memoria

- Historial de preguntas y respuestas.
- Resultados de herramientas.
- Datos persistentes gestionados por la aplicación.

El historial conversacional y el registro de inventario cumplen funciones distintas. Recordar una venta simulada no cambia las existencias.

Autonomía configurable

- Consulta de información.
- Preparación de una propuesta.
- Ejecución con aprobación.
- Ejecución acotada por reglas.

Fuente: OpenAI, Results and state. Consulta: 7 de octubre de 2026.

Problema de integración empresarial

- Un agente de compras necesita consultar el ERP.
- Un agente de soporte necesita leer el CRM.
- Un agente de mantenimiento necesita consultar un sistema de órdenes.
- Cada integración requiere contratos, identidad y permisos.

Necesidad

Ofrecer capacidades reutilizables con una interfaz que las aplicaciones de IA puedan descubrir y utilizar.

Caso propuesto para discusión. Datos y resultados ilustrativos.

MCP: Model Context Protocol

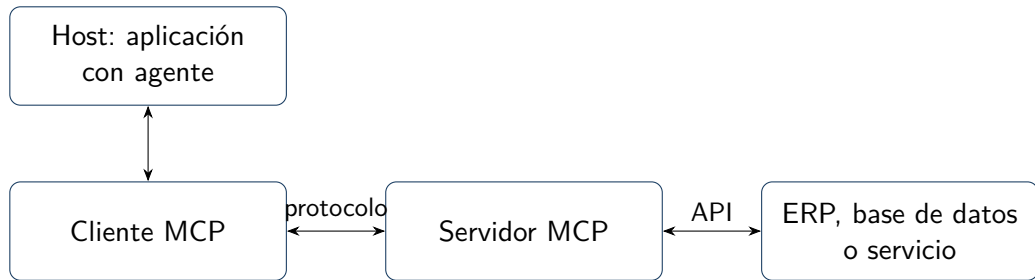
MCP es un protocolo abierto que establece una interfaz común entre aplicaciones de IA y servidores que exponen capacidades externas.

- Estandariza el intercambio de contexto y operaciones.
- Permite descubrir las capacidades que un servidor ofrece.
- Mantiene la integración con los sistemas de negocio en el servidor.

El diseño del agente, la memoria y las reglas del negocio pertenecen a la aplicación.

Fuente: MCP, Architecture overview. Consulta: 7 de octubre de 2026.

Arquitectura de MCP



- El **host** coordina el uso de capacidades.
- El **cliente** intercambia mensajes con el servidor.
- El **servidor** publica capacidades y atiende operaciones.

Fuente: MCP, Architecture overview. Consulta: 7 de octubre de 2026.

Capacidades de un servidor MCP

Capacidad	Propósito	Ejemplo industrial
Tools	Ejecutar funciones	Consultar stock o crear un borrador de orden
Resources	Proporcionar contexto de lectura	Manual de máquina o esquema de datos
Prompts	Ofrecer plantillas reutilizables	Preparar informe de mantenimiento

El soporte de cada capacidad depende del cliente. Una integración que admite herramientas puede tener otro tratamiento para recursos y prompts.

Fuente: MCP, Understanding MCP servers. Consulta: 7 de octubre de 2026.

Interacción con una herramienta MCP

- ❶ El cliente consulta la lista de herramientas: `tools/list`.
- ❷ La aplicación presenta sus descripciones al modelo.
- ❸ El modelo propone herramienta y argumentos.
- ❹ La aplicación aplica permisos y validaciones.
- ❺ El cliente solicita la operación: `tools/call`.
- ❻ El servidor devuelve el resultado y el agente responde.

El servidor puede utilizar una API existente. El modelo no necesita conocer los detalles internos de esa API.

Fuente: MCP, Understanding MCP servers. Consulta: 7 de octubre de 2026.

Herramienta local, API y MCP

Mecanismo	Integración	Situación adecuada
Función Python	La aplicación registra la función	Prototipo de inventario local
API de negocio	Código adaptador llama al servicio	Integración específica con un ERP
Servidor MCP	Publica contratos para clientes compatibles	Capacidad compartida entre aplicaciones

MCP puede envolver una API. La elección depende de la reutilización esperada, la compatibilidad y el costo operativo.

stdio

- Intercambio entre procesos locales.
- Útil para una demostración en el equipo.
- La aplicación administra el proceso.

Una sesión de conversación y una conexión de transporte tienen responsabilidades distintas.

Streamable HTTP

- Comunicación con un servidor remoto.
- Permite desplegar una capacidad compartida.
- Requiere identidad y controles de red.

Fuente: MCP, Architecture overview. Consulta: 7 de octubre de 2026.

- **Agents SDK:** el programa Python administra el agente y sus herramientas.
- **MCP gestionado por el SDK:** la aplicación conecta servidores locales o privados.
- **MCP remoto alojado:** una herramienta de la plataforma permite llamar a un servidor remoto compatible.

Decisión de arquitectura

La conectividad, los permisos y la ubicación de los datos determinan dónde conviene gestionar la conexión.

Fuente: OpenAI, Integrations and observability. Consulta: 7 de octubre de 2026.

Caso: inventario y abastecimiento

Solicitud: “¿Qué productos debemos reponer esta semana?”

- 1 Consultar existencias y mínimos del inventario.
- 2 Calcular faltantes con una regla de negocio.
- 3 Preparar cantidades y costo estimado.
- 4 Obtener aprobación antes de emitir una orden real.

Con MCP: un servidor de inventario expone las consultas y un servidor de compras ofrece la creación de borradores.

Indicadores: tiempo de preparación, exactitud de cantidades y órdenes corregidas por el usuario.

Caso propuesto para discusión. Datos y resultados ilustrativos.

Ejemplo numérico de reposición

Regla: si el stock es menor que el mínimo, comprar hasta el objetivo.

Producto	Stock	Mínimo	Objetivo	Compra	Costo COP
Cuaderno	8	10	20	12	60 000
Lapicero	25	10	30	0	0
Marcador	4	8	15	11	66 000
Memoria USB	4	5	9	5	160 000
Total				28	286 000

Precios unitarios ficticios: 5 000, 2 000, 6 000 y 32 000 COP. El total excluye envío e impuestos.

Caso propuesto para discusión. Datos y resultados ilustrativos.

Caso: mantenimiento industrial

Solicitud: “Revisa las alertas de la bomba B12 y prepara una intervención”.

- Una herramienta consulta mediciones e incidentes históricos.
- Un recurso aporta el manual y los límites operativos.
- El agente organiza evidencias y propone una inspección.
- Un técnico valida el diagnóstico y la orden de trabajo.

MCP: servidor de telemetría y servidor del sistema de mantenimiento.

Indicadores: tiempo de diagnóstico, recomendaciones útiles y falsas alarmas.

Caso propuesto para discusión. Datos y resultados ilustrativos.

Solicitud: “Identifica pedidos con riesgo de retraso y propone alternativas”.

- Consultar pedidos, existencias y ventanas de entrega.
- Invocar un optimizador de rutas con restricciones explícitas.
- Comparar costo, capacidad y tiempo de las alternativas.
- Presentar un plan para aprobación del coordinador.

El optimizador calcula las rutas. El agente integra información y explica las opciones.

Indicadores: entregas a tiempo, costo por pedido y excepciones resueltas.

Caso propuesto para discusión. Datos y resultados ilustrativos.

Caso: soporte y servicio al cliente

Solicitud: “El cliente reporta una falla y solicita una devolución”.

- Consultar pedido e historial del caso en el CRM.
- Recuperar la política de devolución vigente.
- Proponer una respuesta y un siguiente paso.
- Aplicar límites y aprobación a reembolsos.

MCP: capacidades de CRM, pedidos y documentos de políticas.

Indicadores: resolución correcta, tiempo de atención y escalamiento oportuno.

Caso propuesto para discusión. Datos y resultados ilustrativos.

Otros escenarios de aplicación

Área	Tarea del agente	Sistemas conectados
Calidad	Comparar lotes con especificaciones	Laboratorio, documentos, trazabilidad
Energía	Explicar desviaciones de consumo	Medidores, históricos, tarifas
Desarrollo	Investigar una incidencia y proponer cambios	Repositorio, pruebas, seguimiento
Finanzas	Preparar conciliaciones y excepciones	Facturas, contabilidad, reglas

Caso propuesto para discusión. Datos y resultados ilustrativos.

Condiciones favorables

- Datos accesibles y verificables.
- Herramientas con alcance definido.
- Resultado que un experto puede revisar.
- Beneficio medible frente al proceso actual.

Propuesta de adopción: comenzar con consultas y borradores, medir resultados y ampliar el alcance según evidencia.

Condiciones que exigen más diseño

- Datos incompletos o contradictorios.
- Acciones difíciles de revertir.
- Control físico con tiempos estrictos.
- Permisos demasiado amplios.

- Autorizar por usuario, herramienta y recurso.
- Validar argumentos y limitar el acceso a los datos necesarios.
- Mantener credenciales fuera de las instrucciones del modelo.
- Evaluar el origen de servidores, documentos y resultados.
- Registrar operaciones y controlar destinos de conexión.

Los contenidos externos pueden incluir instrucciones maliciosas. La aplicación debe preservar sus reglas y validar las acciones propuestas.

Fuente: MCP, Security Best Practices. Consulta: 7 de octubre de 2026.

Propuesta de diseño para emitir una orden de compra

- ➊ Crear un borrador con productos, cantidades y costo.
- ➋ Mostrar al responsable los datos y la operación concreta.
- ➌ Verificar su autorización en el servicio.
- ➍ Confirmar existencias y precios antes de ejecutar.
- ➎ Usar un identificador para evitar duplicar la orden en un reintento.

Una aprobación debe corresponder a la operación que finalmente se ejecuta. Cambiar cantidades o destinatario exige revisar la autorización.

Evaluación del agente

Dimensión	Evidencia que conviene observar
Exactitud	Datos y cálculos coinciden con el sistema de negocio
Uso de herramientas	Selecciona la operación y los argumentos correctos
Control	Respetar permisos y reconoce datos faltantes
Continuidad	Mantiene contexto sin confundir simulaciones con cambios
Operación	Latencia, costo por tarea y tasa de errores

La traza registra llamadas y resultados. Sirve para evaluar la ejecución sin afirmar que revela el razonamiento interno completo.

Fuente: OpenAI, Integrations and observability. Consulta: 7 de octubre de 2026.

Demostración con el proyecto de inventario

Duración sugerida: 5 a 7 minutos

- ➊ Ejecutar `python main.py` con diagnóstico activo.
- ➋ Preguntar: “¿Qué productos necesitan reposición?”
- ➌ Observar herramienta, argumentos, resultado y respuesta.
- ➍ Pedir un mínimo común de diez unidades y comparar la regla.
- ➎ Simular una venta de siete cuadernos y consultar stock otra vez.
- ➏ Preguntar por una fecha de entrega que el catálogo no contiene.

El proyecto actual utiliza funciones Python locales. Para demostrar MCP se necesita exponer esas funciones mediante un servidor MCP y conectarlo al agente.

Situación: una planta recibe alertas de temperatura y necesita preparar intervenciones.

En equipos, definan:

- Objetivo y criterio de éxito del agente.
- Herramientas, argumentos y resultados esperados.
- Un recurso documental y una plantilla de trabajo.
- Servidores MCP y sistemas a los que se conectan.
- Una acción que requiera revisión humana.

Producto: diagrama de arquitectura y un caso de prueba con resultado esperado.

Caso propuesto para discusión. Datos y resultados ilustrativos.

- Un agente usa información y herramientas para avanzar hacia un objetivo.
- MCP ofrece una interfaz común para capacidades externas.
- Las reglas del negocio y los permisos se implementan en servicios y aplicaciones.
- El valor industrial se demuestra con resultados verificables.
- Un piloto acotado permite evaluar antes de ampliar la autonomía.

Pregunta de discusión: ¿qué tarea de su organización reúne datos accesibles, acciones acotadas y un resultado medible?

Referencias oficiales: agentes y OpenAI

- 1 OpenAI. (s. f.). *Agent definitions*.
developers.openai.com: configuración de agentes
- 2 OpenAI. (s. f.). *Agents SDK quickstart*.
developers.openai.com: tutorial inicial
- 3 OpenAI. (s. f.). *Results and state*.
developers.openai.com: resultados e historial
- 4 OpenAI. (s. f.). *Integrations and observability*.
developers.openai.com: MCP y trazas

Documentación consultada el 7 de octubre de 2026. Los ejemplos industriales son propuestas didácticas, no resultados atribuidos a empresas.

- ① Model Context Protocol. (s. f.). *Architecture overview*.
modelcontextprotocol.io: arquitectura
- ② Model Context Protocol. (s. f.). *Understanding MCP servers*.
modelcontextprotocol.io: capacidades de servidores
- ③ Model Context Protocol. (s. f.). *Security Best Practices*.
modelcontextprotocol.io: seguridad

Las versiones del protocolo y los SDK evolucionan. La compatibilidad concreta se verifica al implementar cada conexión.